Java OOP Laboratory Exercise

Campus Café Ordering System

Name: MANLOSA, JOHN LLOYD
VAILOCES, JOSHUA PAUL

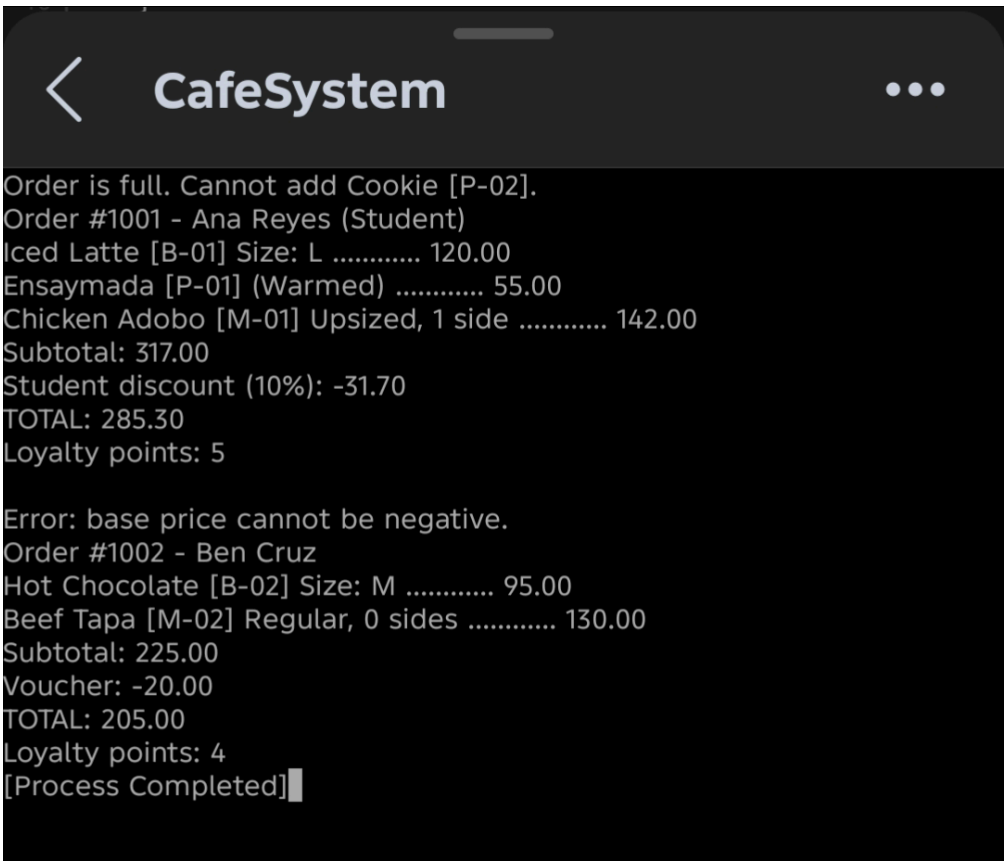
Section: D

Date: November 10, 2026

1. Concept Explanation

if Statement	*Pastry.computePrice()* adds the warming fee and *Meal.computePrice()* adds the upsize fee only when their flag is true.
if-else Statement	Used in *MenuItem.setBasePrice()* to reject negative prices, in *Beverage.computePrice()* for the size price, in *Order.addItem()* for the three-item limit, and in *Order.computeTotal()* for the student discount.
Classes and Objects	Seven classes, each in its own file; *CafeApp.main()* creates the customers, items and orders from them.
Attributes and Behaviors	Every class keeps its data in instance variables and works on it through instance methods.
Object Interaction	*Order* stores a *Customer* and up to three *MenuItem* objects; *printReceipt()* calls each item's methods and adds points to the customer.
Naming Conventions	PascalCase classes, camelCase methods and variables, and UPPER_SNAKE_CASE constants such as *STUDENT_DISCOUNT_RATE*.
Default Constructor	*MenuItem()*, *Beverage()*, *Pastry()*, *Meal()* and *Customer()* set safe starting values.
Parameterized Constructor	Every MenuItem subclass, *Customer* and *Order* accept values at creation so objects are ready to use immediately.
Constructor Overloading	*Customer* has three constructors: no arguments, name only, and name with student status.
public / private / protected	Fields are private, except *name* and *basePrice* in MenuItem which are protected so subclasses can use them; behavior is public.
Getters, Setters, Data Hiding	No field is public. Setters validate input (negative price, invalid size, negative sides, negative points) before changing data.
Method Overloading	*Customer.addPoints(int)* / *addPoints(double)* and *Order.computeTotal()* / *computeTotal(double)* share a name but take different parameters.
Method Overriding	*Beverage*, *Pastry* and *Meal* override *computePrice()* and *getDescription()* with @Override, so *Order* gets the right price through a MenuItem reference.
Abstraction	*MenuItem* is abstract with the abstract method *computePrice()*, so only concrete item types can be created.

2. Program Output



3. Challenges Encountered

- **Matching the loyalty points.** Points must come from the amount paid, not the subtotal. I converted the total to whole points with integer division (285.30 / 50 = 5, 205.00 / 50 = 4).
- **Voucher in the receipt.** *printReceipt()* takes no voucher, so I added an overloaded *printReceipt(double voucher)* and let the no-argument version call it with 0.
- **No ArrayList.** I stored items in three separate fields and filled the first empty slot with an if-else-if chain, keeping an item counter to report a full order.
- **Validating in constructors.** Parameterized constructors reuse the setters so invalid data (like a bad size) keeps a safe default instead of being stored.